

Tidy Survival Analysis: Applying R's Tidyverse to Survival Data

Module 5. Machine Learning

LU MAO

hmao@biostat.wisc.edu

Department of Biostatistics & Medical Informatics

University of Wisconsin-Madison

Aug 3, 2025

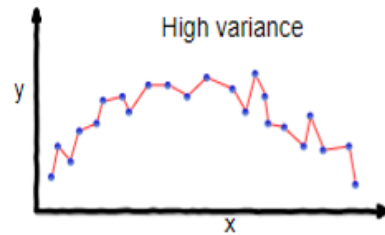
Table of contents

- Machine Learning Survival Models
- `tidymodels` Workflows
- A Case Study
- Summary

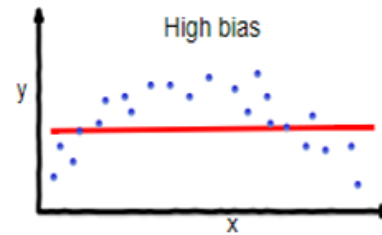
Machine Learning Survival Models

Setting

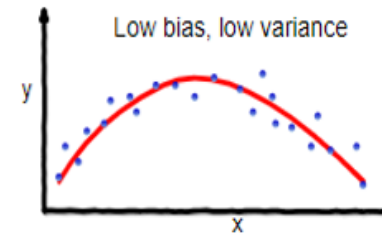
- With many covariates
 - Prediction accuracy: under- vs over-fitting



overfitting



underfitting



Good balance

- Too many predictors → overfitting
- **Interpretation:** easier with fewer predictors

Regularized Cox Regression

- Idea

- Penalize the magnitude of coefficients (L^q -norm) to avoid overfitting

- **Elastic net**: minimize objective function

$$\ell(\beta) = -\log\text{-partial-likelihood}(\beta) + \lambda \sum_{j=1}^p \left\{ \alpha |\beta_j| + 2^{-1}(1 - \alpha) \beta_j^2 \right\}$$

- λ : tuning parameter that controls the strength of penalty
 - Determined by *cross-validation*
 - α : controls the type of penalty
 - $\alpha = 0 \rightarrow$ ridge regression: handles correlated predictors better
 - $\alpha = 1 \rightarrow$ lasso regression: performs variable selection
 - **Implementation**: [glmnet](#) package

Survival Trees

- **Decision trees**

- *Classification and Regression Trees* (CART; Breiman et al., 1984)
- Root node (all sample) $\xrightarrow{\text{covariates}}$ split into (more homogeneous) daughter nodes $\xrightarrow{\text{covariates}}$ split recursively

- **Growing the tree**

- Starting with root node, search partition criteria

$$Z_{.j} \leq z \quad (j = 1, \dots, p; z \in \mathbb{R})$$

for one that minimizes “impurity” (e.g., mean squared deviance residuals) within daughter nodes

$$A = \{i = 1, \dots, n : Z_{ij} \leq z\} \text{ and } B = \{i = 1, \dots, n : Z_{ij} > z\}$$

- Recursive splitting until terminal nodes sufficiently “pure” in outcome

Complexity Control and Prediction

- **Pruning the tree**

- Cut overgrown branches to prevent overfitting
- Penalize number of terminal nodes
- Tune complexity parameter (or minimum size of terminal node)

- **Prediction**

- New $z \rightarrow$ terminal node $\rightarrow$ KM estimates (or median survival)

- **Implementation:** `rpart` package

Random Forests

- **Limitation of a single tree**
 - High variance: small changes in data can lead to large changes in predictions
- **Random forests**
 - Bootstrap samples from training data
 - Take a random subset of covariates to split on (decorrelate the trees)
 - Tune the number of covariates to split on
- **Implementation:** [aorsf](#) package

Model Evaluation

- **Brier score**

- Mean squared error between observed survival status and predicted survival probability
- Inverse probability censoring weighting (IPCW) to account for censoring
- Integrated Brier score: average Brier score over a time interval

- **ROC AUC**

- Area under the receiver operating characteristic (ROC) curve for survival status
- IPCW to handle censoring
- Concordance index: overall AUC over time

tidymodels Workflows

Overview of **tidymodels** and **censored**

- **tidymodels**: a collection of packages for modeling and machine learning in R
 - Provides a *consistent interface* for model training, tuning, and evaluation
 - Key package **parsnip**
 - Supports various model types, including regression, classification, and survival analysis
- **censored**: a **parsnip** extension package for survival data
 - Implements parametric, semiparametric, and tree-based survival models

Data Preparation and Splitting

- Create a **Surv** object as response

- `Surv(time, event)`

```
1 library(tidymodels)
2 library(censored)
3 df <- df |>
4   mutate(
5     surv_obj = Surv(time, event), # create the Surv object as response variable
6     .keep = "unused"             # discard original time and event columns
7   )
```

- Data splitting

- `initial_split()`: splits data into training and testing sets

```
1 df_split <- initial_split(flight_data, prop = 3/4) # default ratio 3:1
2 df_train <- training(df_split) # obtain training set
```

Model Specification

- **Model type**

- `survival_reg()`: parametric AFT models
- `proportional_hazards(penalty = tune())`: (regularized) Cox PH models
- `decision_tree(cost_complexity = tune())`: decision trees
- `rand_forest(mtry = tune())`: random forests

- **Set engine and mode**

- `set_engine("survival")`: for AFT models
- `set_engine("glmnet")`: for Cox PH models
- `set_engine("aorsf")`: for random forests
- `set_mode("censored regression")`: for survival models

```
1 model_spec <- proportional_hazards(penalty = tune()) |> # regularized Cox model (tune lambda)
2   set_engine("glmnet") |> # set engine to glmnet
3   set_mode("censored regression") # set mode to censored regression
```

Recipe and Workflow

- **Recipe:** a series of preprocessing steps for the data
 - `recipe(response ~ ., data = df)`: specify response and predictors
 - `step_mutate()`: standardize numeric predictors
 - `step_dummy()`: convert categorical variables to dummy variables
- **Workflow:** combines model specification and recipe
 - `workflow() |> add_model(model_spec) |> add_recipe(recipe)`

```
1 # Create a recipe
2 model_recipe <- recipe(surv_obj ~ ., data = df_train) |> # specify formula
3   step_mutate(z1 = z1 / 1000) |> # standardize z1
4   step_other(z2, z3, threshold = 0.02) |> # group levels with prop < .02 into "other"
5   step_dummy(all_nominal_predictors()) # convert categorical variables to dummy variables
6 # Create a workflow by combining model and recipe
7 model_wflow <- workflow() |>
8   add_model(model_spec) |> # add model specification
9   add_recipe(model_recipe) # add recipe
```

Tune Hyperparameters

- Cross-validation

- `df_train_folds <- vfold_cv(df_train, v = k)`: create k -folds on training data (default 10)
- `tune_grid(model_wflow, resamples = df_train_folds)`: tune hyperparameters using cross-validation

```
1 # k-fold cross-validation
2 df_train_folds <- vfold_cv(df_train, v = 10) # 10-fold cross-validation
3 # Tune hyperparameters
4 model_res <- tune_grid(
5   model_wflow,
6   resamples = df_train_folds,
7   grid = 10, # number of hyperparameter combinations to try
8   metrics = metric_set(brier_survival, brier_survival_integrated, # specify metrics
9                        roc_auc_survival, concordance_survival),
10  eval_time = seq(0, 84, by = 12) # evaluation time points
11 )
```

Finalize Workflow

- **Examine validation results**

- `collect_metrics(model_res)`: collect metrics from tuning results
- `show_best(model_res, metric = "brier_survival_integrated", n = 5)`: show top 5 models based on Brier score

- **Workflow for best model**

- `param_best <- select_best(model_res, metric = "brier_survival_integrated")`: select best hyperparameters based on Brier score
- `final_wl <- finalize_workflow(model_wflow, param_best)`: finalize workflow with best hyperparameters

```
1 # Extract the best hyperparameters based on Brier score
2 param_best <- select_best(model_res, metric = "brier_survival_integrated")
3 # Finalize the workflow with the best hyperparameters
4 final_wl <- model_wflow |> finalize_workflow(param_best)
```

Fit Final Model

- Fit the finalized workflow

- `final_mod <- last_fit(final_wl, split = df_split)`: fit the finalized workflow on the testing set
- `collect_metrics(final_mod)`: collect metrics of final model on test data

- Make predictions

- `predict(final_mod, new_data = new_data, type = "time")`: predict survival times on new data

```
1 # Fit the finalized workflow on the testing set
2 final_mod <- last_fit(final_wl, split = df_split)
3 # Collect metrics of final model on test data
4 collect_metrics(final_mod) %>%
5   filter(.metric == "brier_survival_integrated")
6 # Make predictions on new data
7 new_data <- testing(df_split) |> slice(1:5) # take first 5 rows of test data
8 predict(final_mod, new_data = new_data, type = "time")
```

A Case Study

GBC: Relapse-Free Survival

- Time to first event

```
1 library(tidymodels) # load tidymodels
2 library(censored)
3 gbc <- read.table("data/gbc.txt", header = TRUE) # Load GBC dataset
4 df <- gbc |> # calculate time to first event (relapse or death)
5   group_by(id) |> # group by id
6   arrange(time) |> # sort rows by time
7   slice(1) |>      # get the first row within each id
8   ungroup() |>
9   mutate(
10     surv_obj = Surv(time, status), # create the Surv object as response variable
11     .after = id, # keep id column after surv_obj
12     .keep = "unused" # discard original time and status columns
13   )
```

Data Preparation

- Analysis dataset

```
1 head(df) # show the first few rows of the dataset
```

```
# A tibble: 6 × 10
```

	id	surv_obj	hormone	age	meno	size	grade	nodes	prog	estr
	<int>	<Surv>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>
1	1	43.836066	1	38	1	18	3	5	141	105
2	2	46.557377	1	52	1	20	1	1	78	14
3	3	41.934426	1	47	1	30	2	1	422	89
4	4	4.852459+	1	40	1	24	1	3	25	11
5	5	61.081967+	2	64	2	19	2	1	19	9
6	6	63.377049+	2	49	2	56	1	3	356	64

- Data splitting

```
1 set.seed(123) # set seed for reproducibility
2 gbc_split <- initial_split(df) # split data into training and testing sets
3 gbc_split
```

```
<Training/Testing/Total>
```

```
<514/172/686>
```

Models to be Trained

- Regularized Cox model

- `proportional_hazards(penalty = tune())`
- Default: $\alpha = 1$ (lasso)
- Tune penalty parameter λ
- Use `glmnet` engine for fitting

- Random forest

- `rand_forest(mtry = tune(), min_n = tune())`
- Tune number of predictors to split on and minimum size of terminal node
- Use `aorsf` engine for fitting

```
1 # Training data
2 gbc_train <- training(gbc_split) # obtain training set
```

Common Recipe

- Recipe for both models

```
1 gbc_recipe <- recipe(surv_obj ~ ., data = gbc_train) |> # specify formula
2   step_mutate(
3     grade = factor(grade),
4     age40 = as.numeric(age >= 40), # create a binary variable for age >= 40
5     prog = prog / 100, # rescale prog
6     estrg = estrg / 100 # rescale estrg
7   ) |>
8   step_dummy(grade) |>
9   step_rm(id) # remove id
10 # gbc_recipe # print recipe information
```

Regularized Cox Model

- Cox model specification and workflow

```
1 # Regularized Cox model specification
2 cox_spec <- proportional_hazards(penalty = tune()) |> # tune lambda
3   set_engine("glmnet") |> # set engine to glmnet
4   set_mode("censored regression") # set mode to censored regression
5 cox_spec # print model specification
```

Proportional Hazards Model Specification (censored regression)

Main Arguments:

```
penalty = tune()
```

Computational engine: glmnet

```
1 # Create a workflow by combining model and recipe
2 cox_wflow <- workflow() |>
3   add_model(cox_spec) |> # add model specification
4   add_recipe(gbc_recipe) # add recipe
```

Model Tuning

- Cross-validation set-up
 - For both models

```
1 set.seed(123) # set seed for reproducibility
2 gbc_folds <- vfold_cv(gbc_train, v = 10) # 10-fold cross-validation
3 # Set evaluation metrics
4 gbc_metrics <- metric_set(brier_survival, brier_survival_integrated,
5                             roc_auc_survival, concordance_survival)
6 gbc_metrics # evaluation metrics info
```

A metric set, consisting of:

- `brier_survival()`, a dynamic survival metric	direction:
minimize	
- `brier_survival_integrated()`, a integrated survival metric	direction:
minimize	
- `roc_auc_survival()`, a dynamic survival metric	direction:
maximize	
- `concordance_survival()`, a static survival metric	direction:
maximize	

```
1 # Set evaluation time points
2 time_points <- seq(0, 84, by = 12) # evaluation time points
```

Cox Model Tuning

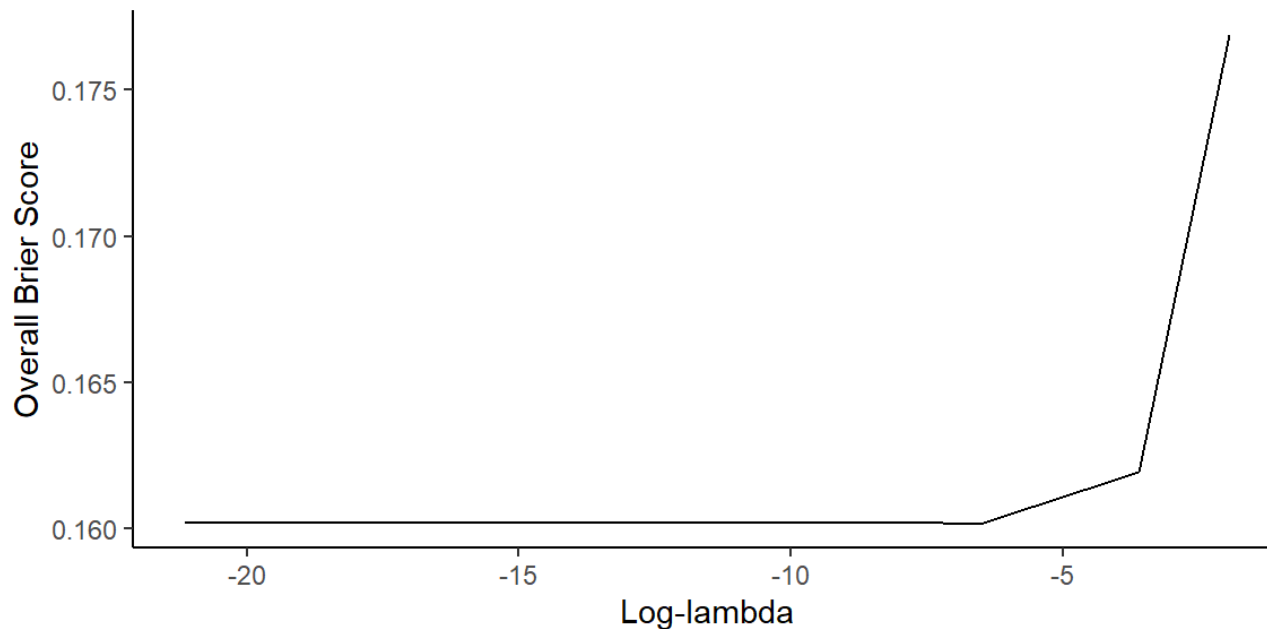
- Tune the regularized Cox model
 - Use `tune_grid()` to perform hyperparameter tuning
 - Evaluate performance using Brier score and ROC AUC

```
1 set.seed(123) # set seed for reproducibility
2 # Tune the regularized Cox model (this will take some time)
3 cox_res <- tune_grid(
4   cox_wflow,
5   resamples = gbc_folds,
6   grid = 10, # number of hyperparameter combinations to try
7   metrics = gbc_metrics, # evaluation metrics
8   eval_time = time_points, # evaluation time points
9   control = control_grid(save_workflow = TRUE) # save workflow
10 )
```

Cox Model Tuning Results

- Plot Brier score as function of $\log \lambda$

```
1 collect_metrics(cox_res) |> # collect metrics from tuning results
2   filter(.metric == "brier_survival_integrated") |> # filter for Brier score
3   ggplot(aes(log(penalty), mean)) + # plot log-lambda vs Brier score
4   geom_line() + # plot line
5   labs(x = "Log-lambda", y = "Overall Brier Score") + # labels
6   theme_classic() # classic theme
```



Best Cox Models

- Show best models
 - Based on Brier score

```
1 show_best(cox_res, metric = "brier_survival_integrated", n = 5) # top 5 models
```

```
# A tibble: 5 × 8
```

	penalty	.metric	.estimator	.eval_time	mean	n	std_err	.config
	<dbl>	<chr>	<chr>	<dbl>	<dbl>	<int>	<dbl>	<chr>
1	0.00147	brier_survival...	standard	NA	0.160	10	0.00775	Prepro...
2	0.0000127	brier_survival...	standard	NA	0.160	10	0.00775	Prepro...
3	0.0000000105	brier_survival...	standard	NA	0.160	10	0.00775	Prepro...
4	0.000754	brier_survival...	standard	NA	0.160	10	0.00775	Prepro...
5	0.00000409	brier_survival...	standard	NA	0.160	10	0.00775	Prepro...

Random Forest Model

- Random forest specification and workflow

```
1 # Random forest model specification
2 rf_spec <- rand_forest(mtry = tune(), min_n = tune()) |> # tune mtry and min_n
3   set_engine("aorsf") |> # set engine to aorsf
4   set_mode("censored regression") # set mode to censored regression
5 rf_spec # print model specification
```

Random Forest Model Specification (censored regression)

Main Arguments:

```
mtry = tune()
min_n = tune()
```

Computational engine: aorsf

```
1 # Create a workflow by combining model and recipe
2 rf_wflow <- workflow() |>
3   add_model(rf_spec) |> # add model specification
4   add_recipe(gbc_recipe) # add recipe
```

Random Forest Tuning

- Tune the random forest model
 - Similar to Cox model tuning

```
1 set.seed(123) # set seed for reproducibility
2 # Tune the random forest model (this will take some time)
3 rf_res <- tune_grid(
4   rf_wflow,
5   resamples = gbc_folds,
6   grid = 10, # number of hyperparameter combinations to try
7   metrics = gbc_metrics, # evaluation metrics
8   eval_time = time_points # evaluation time points
9 )
```

Random Forest Tuning Results

- View validation results

```
1 collect_metrics(rf_res) |> head() # collect metrics from tuning results
```

```
# A tibble: 6 × 9
```

	mtry	min_n	.metric	.estimator	.eval_time	mean	n	std_err	.config
	<int>	<int>	<chr>	<chr>	<dbl>	<dbl>	<int>	<dbl>	<chr>
1	3	30	brier_survival	standard	0	0	10	0	Prepro...
2	3	30	roc_auc_surviv...	standard	0	0.5	10	0	Prepro...
3	3	30	brier_survival	standard	12	0.0635	10	0.00706	Prepro...
4	3	30	roc_auc_surviv...	standard	12	0.827	10	0.0314	Prepro...
5	3	30	brier_survival	standard	24	0.163	10	0.0114	Prepro...
6	3	30	roc_auc_surviv...	standard	24	0.747	10	0.0475	Prepro...

Best Random Forest Models

- Show best models
 - Based on Brier score

```
1 show_best(rf_res, metric = "brier_survival_integrated", n = 5) # top 5 models
```

```
# A tibble: 5 × 9
```

	mtry	min_n	.metric	.estimator	.eval_time	mean	n	std_err	.config
	<int>	<int>	<chr>	<chr>	<dbl>	<dbl>	<int>	<dbl>	<chr>
1	6	24	brier_survival_...	standard	NA	0.155	10	0.00765	Prepro...
2	5	27	brier_survival_...	standard	NA	0.155	10	0.00782	Prepro...
3	4	20	brier_survival_...	standard	NA	0.156	10	0.00777	Prepro...
4	9	36	brier_survival_...	standard	NA	0.156	10	0.00743	Prepro...
5	2	7	brier_survival_...	standard	NA	0.156	10	0.00829	Prepro...

- Conclusion

- Best RF model has lower Brier score than best Cox model

Finalize and Fit Best Model

- Fit final RF model

```
1 # Select best RF hyperparameters (mtry, min_n) based on Brier score
2 param_best <- select_best(rf_res, metric = "brier_survival_integrated")
3 param_best # view results
```

```
# A tibble: 1 × 3
  mtry min_n .config
<int> <int> <chr>
1     6    24 Preprocessor1_Model07
```

```
1 # Finalize the workflow with the best hyperparameters
2 rf_final_wflow <- finalize_workflow(rf_wflow, param_best) # finalize workflow
3 # Fit the finalized workflow on the testing set
4 set.seed(123) # set seed for reproducibility
5 final_rf_fit <- last_fit(
6   rf_final_wflow,
7   split = gbc_split, # use the original split
8   metrics = gbc_metrics, # evaluation metrics
9   eval_time = time_points # evaluation time points
10 )
```

Test Performance (I)

- Collect metrics on test data

```
1 collect_metrics(final_rf_fit) |> # collect overall performance metrics
2   filter(.metric %in% c("concordance_survival", "brier_survival_integrated"))
```

A tibble: 2 × 5

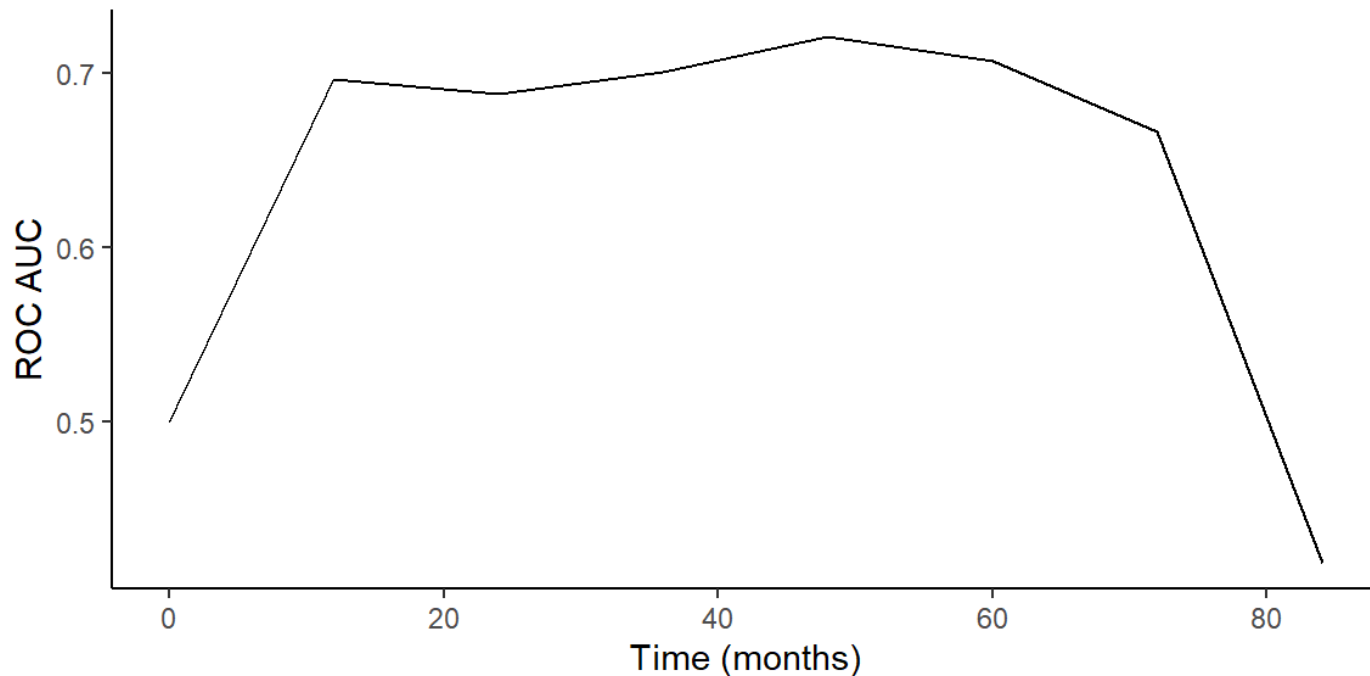
	.metric <chr>	.estimator <chr>	.eval_time <dbl>	.estimate <dbl>	.config <chr>
1	brier_survival_integrated	standard	NA	0.237	Preprocessor1_Model11
2	concordance_survival	standard	NA	0.655	Preprocessor1_Model11

```
1 # Extract test ROC AUC over time
2 roc_test <- collect_metrics(final_rf_fit) |>
3   filter(.metric == "roc_auc_survival") |> # filter for ROC AUC
4   rename(mean = .estimate) # rename mean column
```

Test Performance (II)

- Plot test ROC AUC over time

```
1 roc_test |> # pass the test ROC AUC data
2   ggplot(aes(.eval_time, mean)) + # plot evaluation time vs mean ROC AUC
3   geom_line() + # plot line
4   labs(x = "Time (months)", y = "ROC AUC") + # labels
5   theme_classic()
```



Prediction by Final RF Model

- Extract the fitted workflow
 - Use `extract_workflow()` to get the final model

```
1 gbc_rf <- extract_workflow(final_rf_fit) # extract the fitted workflow
2 # Predict on new data
3 gbc_5 <- testing(gbc_split) |> slice(1:5) # take first 5 rows of test data
4 predict(gbc_rf, new_data = gbc_5, type = "time") # predict survival times
```

```
# A tibble: 5 × 1
```

```
  .pred_time
    <dbl>
1      47.6
2      67.1
3      67.2
4      36.1
5      49.7
```

Cox Model Exercise (I)

- **Task:** extract the best Cox model from `cox_res` and fit it to test data
- Solution

Cox Model Exercise (II)

► Solution - continued

Cox Model Exercise (III)

- **Task:** find the parameter estimates of final Cox model
 - Hint: use `tidy()` function from `broom` package

► Solution

Survival Tree Exercise

- **Task:** fit a survival tree model to the GBC data
 - Use `decision_tree()` with `set_engine("rpart")`
 - Tune complexity parameter `cost_complexity` using `tune()`
 - Use the same recipe as for Cox and RF models
 - Evaluate performance using Brier score and ROC AUC

Summary

Key Takeaways

- **Machine learning:** powerful tools for survival analysis with many covariates
 - Regularized Cox regression, survival trees, and random forests
- **tidymodels:** a consistent interface for modeling and machine learning
 - `parsnip` for model specification and tuning
 - `censored` packages for survival data
 - **Model evaluation:** Brier score and ROC AUC for survival models